

原位测试时训练

Guhao Feng^{1,2,*}, Shengjie Luo^{1,*}, Kai Hua¹, Ge Zhang¹,
Di He^{2,†}, Wenhao Huang^{1,†}, Tianle Cai¹

¹ByteDance Seed, ²Peking University

* 共同一作, † 通讯作者

摘要

静态的“先训练后部署”范式，从根本上限制了 Large Language Models (LLMs) 在面对真实任务中持续涌现的新信息流时，动态调整自身权重的能力。Test-Time Training (TTT) 通过在推理时更新一部分模型参数 (fast weights)，提供了一个很有吸引力的替代思路；然而，它在当前 LLM 生态中的潜力仍受到若干关键障碍的制约，包括架构不兼容、计算效率低，以及 fast weights 的目标函数与语言建模任务不匹配。本文提出 *In-Place Test-Time Training (In-Place TTT)*，这是一个能够无缝赋予 LLMs Test-Time Training 能力的框架。In-Place TTT 将普遍存在的 MLP 模块中的最终投影矩阵视为可适配的 fast weights，从而在无需高成本从零重训的前提下，为 LLMs 提供一种“即插即用”的增强方式。进一步地，我们用一个为自回归语言建模量身设计、并具有理论依据的目标函数，替代了 TTT 中通用的重建目标，使其与 Next-Token Prediction 任务显式对齐。该原则化目标结合高效的分块更新机制，形成了一个高度可扩展、兼容 context parallelism 的算法。大量实验验证了我们框架的有效性：作为一种原位增强方式，它使一个 4B 参数模型在上下文长度最高达 128k 的任务上取得更优性能；而在从零预训练时，它也能稳定优于有竞争力的 TTT 相关方法。消融实验进一步加深了我们对各项设计选择的理解。总体而言，我们的结果表明，In-Place TTT 是迈向 LLM 持续学习范式的一步有前景的探索。

Date:

Correspondence: 联系作者: Di He (dihe@pku.edu.cn) 与 Wenhao Huang (huang.wenhao@bytedance.com)

代码: <https://github.com/ByteDance-Seed/In-Place-TTT>

1 引言

Large Language Models (LLMs) 已经在一系列复杂任务上展现出卓越能力 [8, 11, 39, 51]。这种成功很大程度上建立在静态的“先训练后部署”范式之上：模型先从海量语料中获取知识，随后在推理阶段保持参数固定。然而，这种设计带来了一个根本性限制：模型一旦部署，其权重便无法再更新，因而不能针对输入 token 流所提供的特定上下文进行动态适应。结果是，在测试时，模型处理和推理长时程、持续演化任务的能力会受到限制 [9, 45]，也难以像人类一样从无界的经验流中持续学习 [44]。

In-context learning [8, 56] 通过在上下文中保留所有历史输入 token，为缓解这一问题提供了一种方法。然而，它的有效性受限于模型的 context window，而后者又受到事实标准 attention 机制二次复杂度的约束 [52]。这一瓶颈催生出一系列面向高效扩展 context window 的架构研究 [6, 10, 17, 42]。与之不同，*Test-Time Training (TTT)* 近年来作为一种新范式出现 [3, 47, 48, 53, 63]。TTT 不只是让一个静态模型运行得更高效，而是允许模型动态更新参数并适应任意给定上下文，从而直接针对上述限制。具体而言，TTT 引入了一小部分可在每次新输入到来时即时更新的模型参数，称为 fast weights [43]。通过最小化一个自监督重建目标，这些 fast weights 可以压缩并内化上下文信息，从而充当一种富表达力、在线演化的状态。

尽管概念上很有吸引力，但在当前 LLM 生态中真正释放 TTT 的潜力，仍受到几个关键障碍的阻碍：(i) 现有 TTT 方法通常依赖标准 Transformer block 之外的专门层，这往往需要高成本的从零预训练才能取得理想效果 [47, 48, 53, 67]；(ii) 经典 TTT 机制本质上是串行的 [47, 48]。尽管已有工作探索了分块加速 [3, 30, 49, 63]，但当 TTT 作为主要 token mixer 时，为保持性能往往必须依赖较小的 chunk，从而限制了充分利用现代加速器所需的大规模并行性；(iii) 当前 TTT 在更新 fast weights 时普遍采用通用重建目标，但它并未显式针对支配自回归语言模型的因果式 Next-Token Prediction 任务进行设计，这可能会限制其最终性能。

为弥合这一差距，我们提出 *In-Place Test-Time Training (In-Place TTT)*，这是一个通过直接解决上述障碍、从而无缝赋予 LLMs Test-Time Training 能力的框架。我们的核心洞见是，不再引入新的专门层，而是通过一种 in-place 设计重新利用现有 MLP block (对应障碍 i)。具体来说，In-Place TTT 将 MLP block 中的最终投影矩阵视为其 fast weights，并在推理过程中原位更新它。这样的“即插即用”设计无需修改模型架构，既保留了预训练权重的完整性，也能在无需高成本从零重训的前提下实现在线适应。

为了解决计算低效和目标不匹配问题，我们进一步为语言建模设计了一套专门的适应机制。沿着已有工作 [3, 30, 49, 63, 67] 的思路，我们用可扩展的分块更新规则替代了低效的逐 token 更新 (对应障碍 ii)。此外，我们的 in-place 设计与 attention 机制是互补的。这种协同使得模型不再需要像独立 TTT 层那样依赖小 chunk，从而能在现代加速器上保证高吞吐。与此同时，我们超越了先前工作中的通用重建目标 [48, 67]，提出了一个与 Next-Token Prediction (NTP) 目标显式对齐的新目标函数 (对应障碍 iii)。基于严格的理论分析，我们表明，这一 NTP 对齐目标会促使 fast weights 存储对自回归语言建模真正有预测价值的信息，从而形成一种高效且可扩展的算法。

建立在这些原则性设计选择之上，我们的 In-Place TTT 为增强 LLMs 的动态持续适应能力提供了一个实用且有效的框架。我们在不同算力规模的语言建模任务上进行了大量实验，并将其作为探测模型处理长时程、持续演化任务潜力的现实代理。通过成本相对较低的持续训练，In-Place TTT 使 Qwen3-4B-Base 在上下文长度最高达 128k 的任务上取得更优表现。此外，我们还在最长 32k 的语料上进行从零预训练，将 In-Place TTT 与有竞争力的 TTT 相关方法进行比较，验证了该框架在架构上的优越性。最后，围绕状态大小、chunk 大小以及 fast weight 目标函数的消融实验提供了更深入的认识，确认了每一项设计选择的重要作用。总体而言，我们的结果表明，In-Place TTT 是迈向 LLM 持续学习范式的一步有前景的探索。

2 预备知识：Test-Time Training

本节介绍 Test-Time Training (TTT)。这是一种能够让模型在推理时对新数据进行动态适应的范式 [47, 48, 67]。我们先阐述 TTT 的工作机制，再讨论要将 TTT 成功应用到 LLMs 中所必须满足的关键要求，这也直接构成了我们框架的出发动机。

TTT 机制。TTT 的核心是利用 fast weights [2, 43]，记为 $\mathbf{W}$ 。这些权重构成一个小型神经网络 $f_{\mathbf{W}}(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ ，并在测试时被快速更新。与训练后冻结的标准模型权重不同，fast weights $\mathbf{W}$ 扮演动态记忆的角色，持续地从序列中存储和提取上下文信息。

为了处理输入序列 $\mathbf{x} = [x_1, x_2, \dots, x_N]$ ，通常会先对每个 token $x_i \in \mathbb{R}^d$ 做投影，以得到 TTT 所需的输入，如 query (q_i)、key (k_i) 和 value (v_i)。随后，TTT 机制通过两个核心的串行操作来运行：

1. **更新操作：**更新 fast weights $\mathbf{W}$ ，使 key k_i 与其对应的 value v_i 相关联。形式上，这被写成一步优化过程，用于最小化损失函数 $\mathcal{L}(\cdot, \cdot)$ (例如 Mean Squared Error)，以度量这种关联中的误差。直观地说，这一步会把 (k_i, v_i) 对中的信息编码进神经记忆 $f_{\mathbf{W}}$ 。给定学习率 η ，更新规则为：

$$\mathbf{W}_i \leftarrow \mathbf{W}_{i-1} - \eta \nabla_{\mathbf{W}} \mathcal{L}(f_{\mathbf{W}_{i-1}}(k_i), v_i).$$

2. **应用操作：**使用刚刚更新后的网络 $f_{\mathbf{W}}$ (此时参数为 $\mathbf{W}_i$) 来处理 query q_i ，即 $o_i = f_{\mathbf{W}_i}(q_i)$ 。由于此前 key-value 对中的信息已经编码进 $\mathbf{W}_i$ ，因此输出 o_i 会携带来自历史上下文的增强信息。

尽管上述两步刻画了 TTT 的高层机制，但其具体实现细节可以有很大差异。事实上，近来的大量研究已经探索了丰富的设计空间，包括不同损失函数、更复杂的优化器，以及替代性的神经记忆参数化方式，以提升性能与效率 [3, 5, 32, 54]。这些设计选择会直接影响 fast weights 存储、提取和遗忘序列信息的效果，也决定了 TTT 机制在不同数据模态与任务中的适用性。

LLM 生态中的 TTT 设计要求。尽管 TTT 作为一种动态适应范式很有前景，但在 LLM 生态中真正释放其潜力，仍需解决若干关键挑战。为了成为一个可行且有效的组件，TTT 必须满足以下要求：

- **架构兼容性。**如果一种架构能够从预训练 checkpoint 出发进行 warm start, 我们就称其与 LLM 兼容。但当前的 TTT 机制往往被设计成用于替代 attention 的独立循环层, 而不是与 attention 互补 [29, 47, 48, 53, 67]。这会迫使模型进行高成本的从零预训练, 对主导当前 LLM 生态的数十亿参数大模型而言, 这构成了显著的采用门槛。因此, 一个关键要求是具备“即插即用”的设计, 不需要对原有架构做根本性修改。
- **计算效率。**这一机制必须能在现代并行加速器上高效运行。TTT 的经典逐 token 更新规则本质上是串行的, 因此会严重限制 GPU 和 TPU 的并行处理能力 [3, 47, 48]。这种运行低效使得细粒度更新在高吞吐语言建模场景中并不现实。因此, 高效的 TTT 实现必须超越逐 token 方案并具备可扩展性, 例如采用 chunk-wise 更新机制 [3, 30, 36, 49]。
- **面向语言建模定制的学习目标。**TTT 中占主导地位的自监督目标是重建, 即模型学习关联 (k_i, v_i) 对, 其中 v_i 通常直接由输入 token x_i 本身得到 [29, 47, 48, 53, 67]。尽管这一通用目标能让 TTT 机制存储信息, 但它与语言建模的最终目标, 即预测下一个 token, 之间并不存在天然保证的直接对应。目标值 v 的选择仍是一个关键但尚未被充分探索的设计决策, 对于捕捉 LLM 所需的复杂因果依赖关系而言, 它可能并非最优。

3 原位测试时训练

为了满足 section 2 中提出的设计要求, 我们提出 **In-Place Test-Time Training (In-Place TTT)**, 这是一个旨在为 LLMs 解锁 TTT 能力的框架。我们首先介绍整体框架: 它通过一种原位设计复用已有 MLP block, 从而解决架构不兼容问题, 同时借助分块更新机制保证计算效率 (section 3.1)。随后, 我们详细介绍新的与语言建模对齐的目标函数, 它通过与 Next-Token Prediction (NTP) 目标对齐而专为 LLMs 设计 (section 3.2)。接着, 我们给出该目标优良性质的理论分析 (section 3.3), 最后以实际实现细节收尾 (section 3.4)。

3.1 整体框架

将 MLP Block 重用为原位适应模块。以往 TTT 研究大多将其定位为替代 attention 机制的潜在方案。然而, 这些先前研究通常是在中等规模条件下开展的, 这与现代数十亿参数 LLMs 所处的规模区间相差甚远。因此, 替换 attention 这一核心机制会带来很高的架构风险, 因为 attention 已经学习到的性质对 LLM 的能力至关重要。此外, 引入任何新且随机初始化的层, 也会与 LLM 中已经训练好的数十亿参数形成失衡, 从而需要高成本且往往并不现实的重新训练来消除这种不匹配。

我们的核心洞见是彻底绕开这些挑战。我们不替换、也不新增组件, 而是将一个无处不在的模块, 即 Multi-Layer Perceptron (MLP) block, 重新用作 fast weights。回顾 section 2 中的 TTT 表述, fast weights 的选择本身并没有约束, 也就是说, 任何参数都可以通过 TTT 机制被更新并充当 fast weights。特别地, Transformer 中的 MLP block 也可以被视为一种 key-value memory [22], 在预训练过程中充当承载大量通用知识的“slow weights”。因此, 自然的扩展就是让同一个组件同时承担可适应的“fast weights”角色, 在推理时动态内化短暂的上下文内信息。

形式化地说, 我们采用广泛使用的 gated MLP 架构 [24, 57]。给定隐藏表示 $\mathbf{H}$, gated MLP 的输出表示为 $\mathbf{O} = (\phi(\mathbf{H}\mathbf{W}_{\text{gate}}^\top) \odot (\mathbf{H}\mathbf{W}_{\text{up}}^\top)) \mathbf{W}_{\text{down}}^\top$ 。在我们的框架中, 输入投影 $\mathbf{W}_{\text{up}}$ 和 $\mathbf{W}_{\text{gate}}$ 被视为冻结的 slow weights, 而最终投影矩阵 $\mathbf{W}_{\text{down}}$ 则被重用为可适应的 fast weights。通过只对 $\mathbf{W}_{\text{down}}$ 进行原位更新, 我们保留了模型架构的完整性, 使 TTT 从一种破坏性重构转变为面向 LLMs 的轻量级“即插即用”增强。

借助 Chunk-Wise 更新实现高效适应。除了架构兼容性之外, 我们的原位设计也带来了显著的计算效率优势。传统 TTT 方法由于试图替代 attention 机制, 往往不得不依赖低效的逐 token 更新, 以确保严格因果性并执行细粒度 token mixing。近来的工作已经探索了 chunk-wise 更新来实现加速 [3, 30, 36, 49]。我们的框架同样沿用这一方向, 并进一步彻底绕开这种权衡。由于我们只适配 MLP block, 而保持 attention 层不变, 因此摆脱了逐 token 约束, 可以采用高得多效率的 chunk-wise 更新策略, 并进一步突破小 chunk 的限制 (我们的消融实验结果, 见 4.3 节, 也验证了该框架天然适合 chunk-wise, 尤其是大 chunk 更新, 在 chunk 大小为 512 到 1024 时达到最佳性能)。

具体过程如下。给定中间激活 $\mathbf{Z} = \phi(\mathbf{H}\mathbf{W}_{\text{gate}}^\top) \odot (\mathbf{H}\mathbf{W}_{\text{up}}^\top) \in \mathbb{R}^{n \times d_{\text{ff}}}$, 以及对应的 value 目标和输出 $\mathbf{V}, \mathbf{O} \in \mathbb{R}^{n \times d_{\text{model}}}$, 我们将它们划分为 k 个大小为 C 、互不重叠的 chunk, 记作 $\square_{[i]} = \square_{iC+1:(i+1)C} \in \mathbb{R}^{C \times d'}$, 其中 $\square \in \{\mathbf{Z}, \mathbf{V}, \mathbf{O}\}$, $i \in [k]$, d' 为对应维度。记 $\mathbf{W}_{\text{down}}^{(i)}$ 为处理第 i 个 chunk 之前的 fast weights 状态, 且 $\mathbf{W}_{\text{down}}^{(0)} = \mathbf{W}_{\text{down}}$ 。对每个 chunk $i \in [k]$, 我们执行两个串行操作:

1. **应用操作:** 使用当前状态的 fast weights $\mathbf{W}_{\text{down}}^{(i)}$ 处理 chunk $\mathbf{Z}_{[i]}$, 即 $\mathbf{O}_{[i]} = \mathbf{Z}_{[i]}(\mathbf{W}_{\text{down}}^{(i)})^\top$ 。

2. **更新操作**: 以 $\mathbf{Z}_{[i]}$ 作为 keys、 $\mathbf{V}_{[i]}$ 作为 values, 对 fast weights $\mathbf{W}_{\text{down}}^{(i)}$ 进行更新。该更新通过一次带损失函数 $\mathcal{L}$ 与学习率 η 的梯度下降步骤完成: $\mathbf{W}_{\text{down}}^{(i+1)} = \mathbf{W}_{\text{down}}^{(i)} - \eta \nabla_{\mathbf{W}} \mathcal{L}(\mathbf{Z}_{[i]}(\mathbf{W}_{\text{down}}^{(i)})^\top, \mathbf{V}_{[i]})$ 。

这种 chunk-wise 更新策略与已有尝试类似, 都是面向现代硬件设计的 [3, 30, 36, 49]。此外, 由于我们采用的是原位 MLP 适配, 因此可以使用较大的 chunk 大小 C 一次处理大块 token, 从而更充分地利用并行性以及 GPU 或 TPU 的算力。

3.2 与语言建模对齐的目标函数

在高效的原位适应框架建立之后, In-Place TTT 的性能关键就转移到了学习目标的设计上。在本小节中, 我们介绍与 Language Modeling 对齐的目标函数, 它是专门为 LLMs 量身设计的。

以往 TTT 方法通常采用重建目标, 例如 $\mathcal{L}(f_{\mathbf{W}}(k), v)$, 其中 k 和 v 都是同一个输入 token x 的线性投影输出 [47, 48, 67]。这种做法鼓励模型去记忆当前 token 的表示。我们认为, 对于语言建模任务而言, 这并不是最优选择。相反, 我们提出将目标函数与支配 LLMs 的 Next-Token Prediction (NTP) 目标对齐。

为此, 我们将目标 v 设计为包含未来 token 信息。形式化地, 我们定义目标为 $\hat{\mathbf{V}} = \text{Conv1D}(\mathbf{X}_0)\mathbf{W}_{\text{target}}$, 其中 $\mathbf{X}_0 \in \mathbb{R}^{n \times d_{\text{model}}}$ 表示 token embedding, $\text{Conv1D}(\cdot)$ 是 1D Convolution 算子, 而 $\mathbf{W}_{\text{target}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ 是一个可训练的投影矩阵。在这一表述下, 我们可以控制目标 $\hat{\mathbf{V}}$ 中包含多少未来 token 信息。例如, 只需将 $\mathbf{W}_{\text{target}}$ 参数化为恒等变换, 并将 $\text{Conv1D}(\cdot)$ 的卷积核权重设置为对下一个 token 为 1、对其他 token 为 0, 就可以实现 Next-Token 目标。

有了这一对齐目标后, 为了简洁起见, 我们采用广泛使用的相似性度量来实例化损失函数, 即 $\mathcal{L}(\cdot, \cdot) = -\langle \cdot, \cdot \rangle_F$ 。在该损失函数下, 我们的 chunk-wise 机制中关于 fast weights 的梯度可以直接推导为:

$$\mathbf{W}_{\text{down}}^{(i)} = \mathbf{W}_{\text{down}}^{(i-1)} + \eta \hat{\mathbf{V}}_{[i]}^\top \mathbf{Z}_{[i]}. \quad (1)$$

3.3 理论分析

直观来看, 我们与 LM 对齐的目标会显式鼓励 fast weights 压缩对未来 token 预测有用的信息, 从而提升模型的动态适应能力。在本小节中, 我们通过理论分析将这一直觉形式化, 说明该目标带来的收益。我们的分析建立在经典的 *induction head* 场景之上 [19, 38], 这一机制被认为是 LLMs 中 in-context learning 的关键基础之一。

设定。考虑一个输入序列, 其中某个 key-value 对 $(x_{t^*}, x_{t^*+1}) = (k^*, v^*)$ 出现在任意位置 t^* 。随后, 在某个 query 位置 $n > t^*$ 处, key k^* 再次出现, 即 $x_n = k^*$ 。此时模型必须正确预测与之关联的 value, 即 $x_{n+1} = v^*$ 。

不失一般性地, 我们分析一个经过 In-Place TTT 增强的单层 Transformer block。记 $\mathbf{Z}_t \in \mathbb{R}^{d_{\text{ff}}}$ 为 token x_t 的中间激活, $\mathbf{E}_w \in \mathbb{R}^{d_{\text{model}}}$ 为 token x_w 的 embedding。在我们的框架中, 来自先前上下文 chunk 的 fast weights 更新为 $\Delta \mathbf{W}_{\text{down}} = \eta \sum_{t \in \text{prior chunks}} \mathbf{V}_t \mathbf{Z}_t^\top$ 。这一更新会使 query 位置 n 上的输出 logit 发生变化, 记为 $\Delta \ell_n[w] = \mathbf{E}_w^\top \Delta \mathbf{W}_{\text{down}} \mathbf{Z}_n$ 。我们比较 TTT 目标 $\mathbf{V}_t$ 的两种选择:

- 重建目标: $\mathbf{V}_t^{\text{rec}} = \mathbf{E}_{x_t}$, 即当前 token 的 embedding。
- LM 对齐目标: $\mathbf{V}_t^{\text{LM}} = \mathbf{E}_{x_{t+1}}$, 即下一个 token 的 embedding。

假设。我们的分析建立在关于 embedding 和中间激活性质的两个温和假设之上, 这些假设在 Transformer 的理论分析中都很常见:

1. *Embedding* 的近似正交性: 对任意两个不同的 token $w_i, w_j \in \mathcal{V}$, 它们的 embedding 几乎正交: $|\mathbf{e}_{w_i}^\top \mathbf{e}_{w_j}| \leq \epsilon$, 其中 $\epsilon > 0$ 是一个较小常数。此外, embedding 还具有非平凡的范数: 存在常数 c_{norm} 使得 $\|\mathbf{e}_{w_i}\|_2^2 \geq c_{\text{norm}}^2 > 0$ 。
2. *Key-Query* 对齐: query token $x_n = k^*$ 的中间激活 $\mathbf{Z}_n$ 与其对应 key token $x_{t^*} = k^*$ 的中间激活 $\mathbf{Z}_{t^*}$ 是对齐的, 即 $\mathbb{E}[\mathbf{Z}_{t^*}^\top \mathbf{Z}_n] = c_{\text{align}} > 0$ 。而对其他位置 $t \neq t^*$, 对应 token 与该 query 无关, 即 $\mathbb{E}[(\mathbf{V}_t \mathbf{Z}_t^\top) \mathbf{Z}_n] = 0$ 。

在这一设定下, 我们给出主要理论结果:

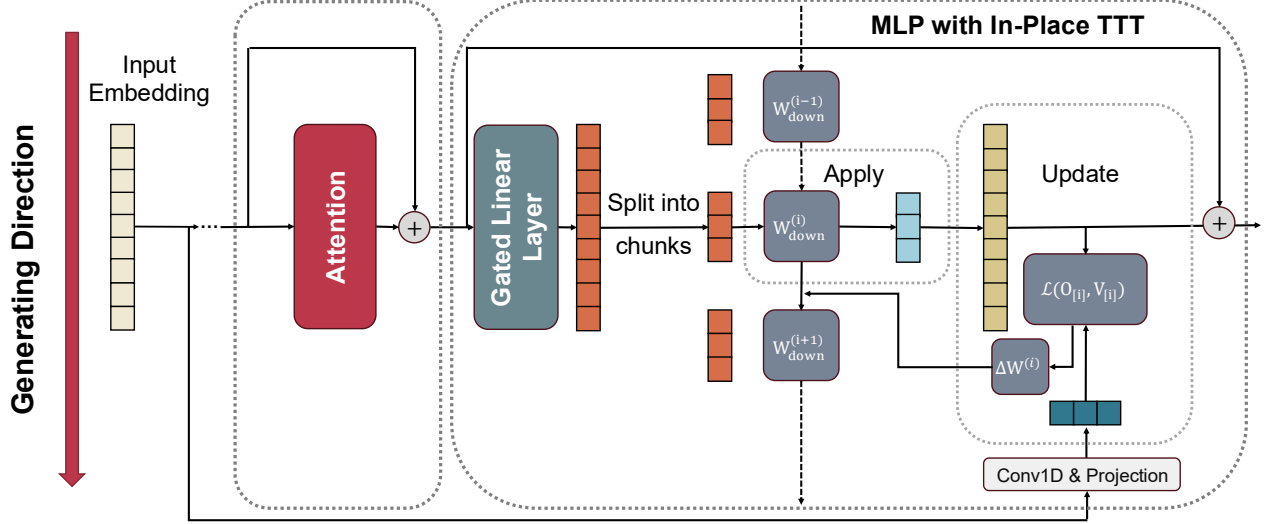


Figure 1 我们的 In-Place Test-Time Training 整体框架。该模块以串行方式处理输入 chunk。对于每个 chunk，先将当前 fast weights 应用到中间激活 $\mathbf{Z}$ 上以生成输出；随后，再利用激活 $\mathbf{Z}$ 以及由 token embedding 导出的 value $\mathbf{V}$ 对这些权重进行更新。这种“先应用、后更新”的循环使模型能够以严格因果的方式对不断到来的上下文进行动态适应。

Theorem 1 (LM 对齐目标与重建目标在 logit 层面的效应对比). 在上述设定与假设下，对任意学习率 $\lambda_{lr} > 0$ ，使用 LM 对齐目标进行一次更新后，logits 的期望变化 $\Delta \ell_n$ 满足：

$$(\text{正确 token 的 logit 增大}) \quad \mathbb{E}[\Delta \ell_n[v^*]] \geq \lambda_{lr} \cdot c_{norm}^2 \cdot c_{align}, \quad (2)$$

$$(\text{其他 token 的 logit 几乎不变}) \quad |\mathbb{E}[\Delta \ell_n[w]]| \leq \lambda_{lr} \cdot \epsilon \cdot c_{align}, \quad \forall w \neq v^*. \quad (3)$$

相反，对于重建目标，正确 token 的 logit 期望变化可以忽略不计： $|\mathbb{E}[\Delta \ell_n[v^*]]| \leq \lambda_{lr} \cdot \epsilon \cdot c_{align}$.

证明见附录 A。根据 theorem 1，LM 对齐目标在期望意义下能够保证正确下一个 token v^* 的 logit 增大，同时使其他 token 的 logit 基本保持不变，因此可以直接帮助模型完成预测任务。相比之下，重建目标并不具备这样的预测收益，无法提升正确 token 的 logit。在实际实现中，我们把这一原则从单个下一个 token 扩展为对未来多个 token 的局部可学习组合，这也与先进 LLM 中近期关于 Multi-Token Prediction 的积极结果相一致 [37]，可被视为 NTP 目标的一种有效扩展。这样一来，In-Place TTT 就能捕捉更丰富的预测信号，从而比简单重建更有效地压缩有用的上下文信息。

3.4 实现细节

综合前述设计，figure 1 展示了我们的 In-Place TTT 框架。下面我们进一步说明实际实现细节，这些设计都是围绕现代硬件上的高效率与可扩展性展开的。特别地，我们的方法与 Context Parallelism (CP) 完全兼容，它依赖并行 scan 算法在同时处理序列 chunk 的同时，仍保留自回归更新所要求的严格因果语义。我们也给出进一步讨论。

基于 Context Parallelism 的高效实现 equation (1) 中更新规则的可结合性，使 In-Place TTT 天然适合 context-parallel 实现：它沿序列长度切分序列，并同时处理这些 chunk。整个过程分为三个阶段：(i) 对所有 chunk $i \in \{1, \dots, T\}$ ，并行计算中间激活 $\mathbf{Z}_{[i]}$ 以及 fast weight 更新量 $\Delta \mathbf{W}_{\text{down}}^{(i)} = (\hat{\mathbf{V}}_{[i]})^\top \mathbf{Z}_{[i]}$ ；(ii) 对序列 $[\dots, \Delta \mathbf{W}_{\text{down}}^{(i)}, \Delta \mathbf{W}_{\text{down}}^{(i+1)}, \dots]$ 做一次 prefix sum，以计算每个 chunk 的累计更新 $\Delta \mathbf{S}_i = \sum_{j=1}^{i-1} \Delta \mathbf{W}_j$ ，这一过程在现代加速器上可以非常高效；(iii) 并行计算每个 chunk 对应的有效 fast weights， $\mathbf{W}_{\text{down}}^{(i-1)} = \mathbf{W}_{\text{down}}^{(0)} + \eta \Delta \mathbf{S}_i$ ，以及相应输出 $\mathbf{O}_{[i]} = \mathbf{Z}_{[i]}(\mathbf{W}_{\text{down}}^{(i-1)})^\top$ 。

因果性与边界处理。为了确保 chunk i 自身的更新增量不包含未来信息，我们在生成 value 时对 1D convolution 使用 causal padding。这样每个 delta 的计算就被限制在各自的 chunk 内部，使并行 scan 在数学上等价于顺序更新。此外，在文档边界处，我们会将 fast weights 重置回预训练状态，以避免不同独立序列之间的上下文泄漏。最终的 context parallel 算法见附录 B 中的 algorithm 1。

讨论。总的来说，我们对 In-Place TTT 的实现，将一个简单且计算高效的更新规则与并行 scan 算法协同结合。借助精心设计的边界与 padding 管理，这一设计不仅让方法足够快、足够可扩展，而且在数学上等价于严格因果

的顺序过程。最终得到的模块是 *CP-native*、完全因果的，并且能够无缝地作为标准 Transformer 架构中 MLP block 的“即插即用”替代增强。最后还值得注意的是，我们框架的核心原则与具体损失函数及其优化器的选择是正交的，而这些方向在更广泛的 TTT 文献中已经被大量研究；我们将其留作未来一个很有前景的探索方向。

4 实验

本节通过一系列实验，从经验上验证 In-Place TTT 框架的有效性。具体来说，我们希望回答以下研究问题：

- **Q1**: In-Place TTT 作为一种“即插即用”增强，能在多大程度上提升预训练 LLMs？
- **Q2**: 当从零开始训练时，In-Place TTT 与以往 TTT 方法相比表现如何？
- **Q3**: In-Place TTT 框架中的关键设计选择分别带来哪些影响？

我们以不同规模的语言建模任务作为现实代理，在后续小节中通过精心设计的实验回答这些问题。受篇幅限制，实验设定的详细描述放在附录 C 中。

Table 1 RULER [28] 上的评测结果。报告平均准确率 (%), 最佳结果以 **粗体** 标出。

模型	域内评测						外推
	4k	8k	16k	32k	64k	128k	256k
Mistral-7B [31]	93.6	91.2	87.2	75.4	49.0	13.8	-
GLM3-6B [23]	87.8	83.4	78.6	69.9	56.0	42.0	-
Phi3-medium-14B [1]	93.3	93.2	91.1	86.8	78.6	46.1	-
Llama3-8B [41]	92.8	90.3	85.7	79.9	76.3	69.5	-
Qwen3-4B (Instruct) [57]	95.1	93.6	91.0	87.8	77.8	66.0	-
Baseline	96.6	94.1	92.1	88.7	74.3	74.8	41.7
In-Place TTT	96.1	95.6	92.7	89.3	78.7	77.0	43.9

4.1 In-Place TTT 作为预训练 LLM 的即插即用增强

为了验证 In-Place TTT 能否作为一种“即插即用”增强用于现有预训练 LLMs，我们从具有竞争力的开源模型 *Qwen3-4B-Base* 出发。该模型原始的 context window 为 32k，因此我们可以通过不同上下文长度的语言建模任务，来模拟那些需要 Test-Time Training 能力的长时程、持续演化任务。具体而言，我们比较两种模型的性能：(1) *Qwen3-4B-Base* (Baseline)；(2) *Qwen3-4B-Base + In-Place TTT* (In-Place TTT)。两者经历完全相同的持续训练流程，从而保证比较公平，使 In-Place TTT 成为唯一变量。

训练与评测。持续训练流程分为两个阶段：第一阶段使用约 ~20B token、上下文长度为 32k；第二阶段使用约 ~15B token、上下文长度为 128k。训练数据集的详细描述见附录 C.1。为了有效处理这些长序列，我们借助 YaRN [42] 对模型的 Rotary Position Embeddings 进行适配。我们在流行的 OpenCompass 框架 [13] 中，使用 RULER 基准 [28] 评测两个模型的长上下文性能，评测长度从 4k 到 256k 不等。其中 256k 设置专门衡量模型超出 128k 上下文长度上限时的外推能力。训练细节的完整描述见附录 C.2。

结果与讨论。table 1 总结的结果表明，In-Place TTT 能显著提升预训练模型的长上下文能力。尤其可以清晰地看到一个趋势：虽然两种模型在短上下文下都具有竞争力，但随着序列长度增加，加入 In-Place TTT 的 *Qwen3-4B-Base* 会形成持续且不断扩大的优势。它在 64k 和 128k 上下文长度上都取得了明显提升。更关键的是，这种优势在外推到 256k 上下文时依然成立，说明其具备更强的泛化能力。这些发现证明，In-Place TTT 可以无缝集成到预训练 LLM 中，以增强其长上下文处理能力。该模型在接近和超出原始上下文长度范围时的强表现，也验证了我们的方法是扩展现有 LLM 能力的一种实用而强大的工具。

扩展到更多模型。为了验证方法的通用性，我们进一步按照相同的持续训练协议，将 In-Place TTT 作为即插即用增强应用到另外两个模型上：LLaMA-3.1-8B [24] 和 *Qwen3-14B-Base* [57]。如表 2 所示，In-Place TTT 在两个模型的所有上下文长度上都稳定提升了 RULER 分数，且在较长上下文下收益尤为明显，例如 LLaMA-3.1-8B 在 64k 上提升 +2.1，*Qwen3-14B-Base* 在 64k 上提升 +2.7。此外，当与 YaRN [42] 结合进行位置外推时，这种

Table 2 In-Place TTT 在 RULER 基准上扩展到 LLaMA-3.1-8B 和 Qwen3-14B-Base 的结果。我们报告平均准确率 (%), 最佳结果以 **粗体** 标出。

基础模型	方法	4k	8k	16k	32k	64k	64k+YaRN
LLaMA-3.1-8B	Baseline	93.9	92.1	92.5	91.1	81.6	–
	In-Place TTT	94.4	93.0	93.3	91.7	83.7	–
Qwen3-14B	Baseline	96.8	95.0	94.6	90.7	67.9	81.3
	In-Place TTT	97.2	95.7	95.2	91.2	70.6	82.5

提升依旧存在, 说明我们的方法与 RoPE 扩展技术是正交的。这些跨越不同模型家族和规模 (4B–14B) 的结果进一步说明, In-Place TTT 是一种广泛适用于预训练 LLMs 的即插即用增强方案。

4.2 从零预训练: 对比分析

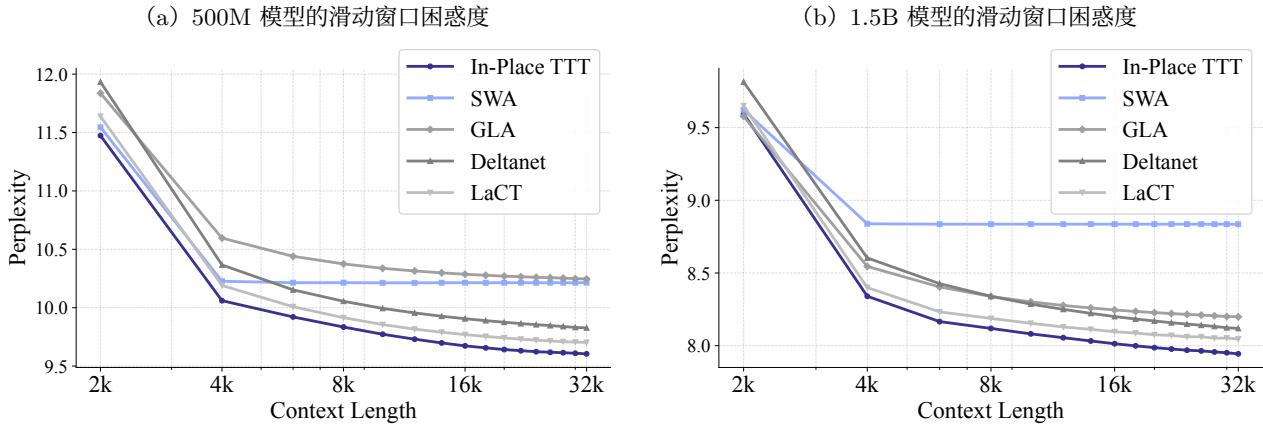


Figure 2 500M (左) 和 1.5B (右) 参数模型在 Pile 数据集上、不同上下文长度下的滑动窗口困惑度。我们的 In-Place TTT 在各个设置下都稳定取得低于所有竞争基线的困惑度。

模型架构	常识推理					长上下文评测		
	HellaSwag	ARC-E	ARC-C	MMLU	PIQA	RULER-4k	RULER-8k	RULER-16k
基线	Full Attn.	55.67	64.52	33.19	36.43	72.63	45.77	38.09
	SWA	54.92	64.18	32.85	36.06	72.58	14.77	9.91
I.P. TTT	Full Attn.	55.85	64.98	32.34	37.42	73.29	49.98	43.82
	SWA	55.24	64.60	33.70	36.48	72.03	28.33	26.80

Table 3 4B 模型在常识推理与长上下文评测基准上的结果。最佳性能以 **粗体** 标出。其中, “SWA” 表示 Sliding-Window Attention, “Full Attn.” 表示 Full Attention, “I.P. TTT” 表示我们的 In-Place TTT。

在展示了 In-Place TTT 作为“即插即用”模块的有效性之后, 我们进一步评估它在从零预训练模型中的性能与可扩展性。我们的分析分两步进行: 首先验证它在 500M 和 1.5B 规模上的语言建模能力, 然后评估它在更大规模 4B 模型上的可扩展性与影响。

实验设置。 首先, 在 500M 和 1.5B 参数规模上, 我们基于 TogetherAI [50], 将 In-Place TTT 与已有 TTT 相关方法以及高效 attention 方法进行基准比较。参与比较的竞争性基线包括: (1) 带 sliding window attention (SWA) 的标准 Transformer [6, 10]; (2) Gated Linear Attention (GLA) [60]; (3) DeltaNet [43, 59, 62]; (4) Large Chunk Test-Time Training (LaCT) [67]。为了公平比较, In-Place TTT 与 LaCT 都建立在 SWA 骨干之

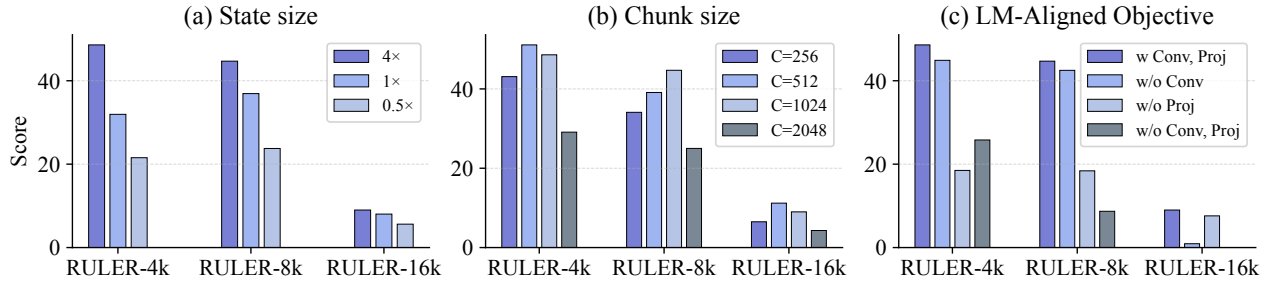


Figure 3 In-Place TTT 框架关键设计选择的消融实验，基于 1.7B 参数模型在 RULER 基准上进行评测。图中展示了以下因素的影响：(a) 状态大小，说明性能会随状态规模增大而提升；(b) chunk 大小，展示了性能与效率之间的权衡，其中中等大小（如 512、1024）最优；(c) LM 对齐的 Value 目标，说明卷积（w Conv）和投影（w Proj）二者都至关重要。

上。所有模型都在上下文长度为 32k 的序列上训练。

在这些结果的基础上，我们进一步扩展到 4B 参数模型，以评估 In-Place TTT 方法的可扩展性。具体来说，我们比较了带 Full Attention 的 Transformer 和带 SWA 的 Transformer，与其各自加入 In-Place TTT 增强后的对应版本。这些模型使用 8k 上下文长度训练了 120B token。关于数据集、模型配置和训练流程的详细说明见 sections C.1 to C.3。

评测。对于 500M 和 1.5B 模型，我们在由 Pile [20] 与 Proof-Pile-2 [40] 构成的验证集上，使用 *Sliding Window Perplexity* 评估其长上下文利用能力。该指标衡量的是：在不断扩展前置上下文时，固定末尾一段 token 上的困惑度；如果困惑度随上下文增长而下降，就说明模型有效利用了更多上下文。对于 4B 模型，我们则在更广泛的下游任务上进行评测，包括常识推理基准 (HellaSwag [66]、ARC [12]、MMLU [26, 27]、PIQA [7]) 以及长上下文 RULER 基准 [28]。

结果与讨论。在图 2 中，我们绘制了 500M/1.5B 模型的滑动窗口困惑度随上下文长度变化的曲线。可以清楚看到，我们的 In-Place TTT 在所有设置下都稳定取得低于竞争基线的验证困惑度，并且其性能会随着上下文增长持续改善，直到完整的 32k 上下文。这种持续改进说明，其核心机制确实成功压缩并利用了不断到来的上下文信息。

此外，表 3 的结果进一步表明，无论是 Full Attention 还是 SWA 的 4B 参数 Transformer，在大多数常识推理任务上都能因 In-Place TTT 获得稳定提升。与此同时，加入 In-Place TTT 的模型在长上下文评测中表现更优，例如带 Full Attention 的 Transformer 在 RULER-16k 上从 6.58 提升到 19.99，带 SWA 的模型在 RULER-8k 上从 9.91 提升到 26.80。这些显著收益，尤其是在不同规模模型上都成立，说明 In-Place TTT 是一种高效且具备良好可扩展性的方法。

4.3 消融实验：关键设计选择的影响

最后，我们在一个 1.7B 参数模型上基于 RULER 进行一系列消融实验，以更深入地理解各项设计选择。详细设定见附录 C.3 和 C.2。

状态大小的影响。我们首先研究性能如何随 fast weights 大小变化，而这一大小可通过调整启用 TTT 的层数来控制。图 3 (a) 清楚展示出一个趋势：随着状态大小扩大，In-Place TTT 的性能会稳定提升。这说明更大的 fast weights 能让模型更有效地适应上下文信息，也进一步支持了我们复用大量 MLP 状态的设计思路。

Chunk 大小的影响。section 3.1 中的 chunk 大小 C 同时控制 fast weights 更新的粒度与并行程度，因此体现出效率与性能之间的权衡。通过改变 chunk 大小，图 3 (b) 表明， $C = 512$ 与 $C = 1024$ 相较于其他选择都能取得更优性能，而 $C = 1024$ 具有更好的效率。

LM 对齐目标的影响。接下来，我们深入分析 section 3.2 中量身定制的 LM 对齐目标，其中目标被定义为 $\hat{\mathbf{V}} = \text{Conv1D}(\mathbf{X}_0)\mathbf{W}_{\text{target}}$ 。其中，Conv1D 用来生成包含未来 token 信息的目标，而 $\mathbf{W}_{\text{target}}$ 则承担投影变换作用。在图 3 (c) 中，我们系统地消融了这些组件的不同组合。结果表明，这两个组件对于保证性能都不可或缺；其中 Conv1D 在长上下文上作用尤为关键，而 $\mathbf{W}_{\text{target}}$ 对短上下文则更重要。这些结果与 section 3.3 中的理论分析相一致，强有力地支持了我们为语言建模推导定制目标函数的动机。

In-Place TTT 的效率影响。我们进一步研究 In-Place TTT 所带来的计算开销。在 figure 4 中，我们比较了加入与未加入 In-Place TTT 时的 prefill 吞吐量和内存消耗。结果验证了我们实际实现的高效率。

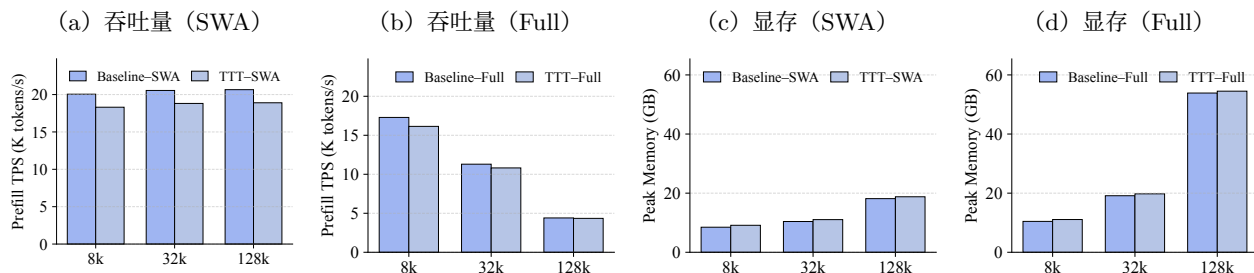


Figure 4 In-Place TTT 的效率分析。图中给出了 4B 模型在不同上下文长度下，采用 Sliding-Window Attention (SWA) 与 Full Attention 时的 prefill 吞吐量 (a, b) 和峰值显存 (c, d)。在实际场景中，我们的 In-Place TTT 几乎不会引入额外开销。

5 相关工作

Test-Time Training (TTT)。Test-Time Training (TTT) 是一种在推理阶段通过更新一小部分参数，即 *fast weights* [2]，从而使模型能够对持续数据流进行动态适应的范式。TTT 最初在计算机视觉中取得成功 [47, 53]，此后又被扩展到众多其他模态，包括语言 [48]、视频 [15] 和音频 [18]，体现出其广泛适用性。这一方向的研究主要集中在两个方面以提升 TTT 的效果：设计更复杂的测试时优化器 [4]，以及提出新的自监督在线学习目标 [3, 32]。然而，由于其天然串行的逐 token 更新过程，TTT 的计算效率始终是一个关键瓶颈。chunk-wise Test-Time Training 框架首次通过引入分块更新机制来直接应对这一挑战，从而更好地利用并行硬件 [3, 30, 36, 49, 63, 67]。尽管如此，由于 TTT 仍承担主要 token mixer 的角色，为了维持性能，它仍需依赖较小的 chunk，而这又反过来限制了充分发挥现代加速器所需的大规模并行性。此外，先前工作并未解决如何将 TTT 无缝整合到大型预训练模型中，也未为 LLM 的自回归特性设计专门的学习目标，而这些正是本文直接应对的空白。

高效长上下文架构。另一条平行研究路线试图通过缓解标准 attention 机制的二次复杂度，来扩展 LLMs 的有效 context window。主要方法包括：1) Sparse attention 方法，通过滑动窗口、跨步窗口等固定模式限制 token 之间的交互范围 [6, 10, 65]；2) 线性时间变体，通过近似 attention 机制，或者用高效的循环式/门控式形式替代 attention，例如 linear attention [33, 43] 与 Gated Linear Attention (GLA) [58]；3) State-Space Models (SSMs)，它将序列历史压缩到紧凑的隐状态中，从而实现线性复杂度处理 [16, 17]。近期，*delta rule* 已经成为 linear attention 和 SSMs 中一种流行的设计选择，因为它兼具更强表达能力和高度并行化实现 [61]。这些架构进展与我们的框架是互补的。它们关注的是如何高效地处理长上下文，而 TTT 提供的是一种对其中信息进行在线适应的机制。由于这些高效骨干同样包含 MLP block，我们的 In-Place TTT 也能够自然地集成进去。我们将这一方向留作未来工作。

记忆设计与增强。另一个相关研究领域是通过为神经架构增添显式记忆模块，来增强其推理与上下文理解能力。这些方法大致可按功能区分：有些旨在把持久的、与任务无关的知识存入外部记忆库；另一些则聚焦于捕捉来自即时上下文的、短暂且依赖数据的信息 [25, 34, 35, 55, 64]。后者，即上下文记忆，已经通过多种机制实现，包括循环状态转移、Transformer 中基于 attention 的上下文聚合 [14]，以及对 *fast weights* 的快速、基于梯度的更新 [61]。Test-Time Training (TTT) 正是这一类别中的一个强有力实例，它在概念上扩展了 Recurrent Neural Networks (RNNs) 中隐藏状态的思想。TTT 不再把上下文历史压缩到固定大小的激活向量中，而是指定模型自身的一部分参数，即 *fast weights*，充当高容量的动态记忆 [2, 43]。这些权重会在推理时即时更新，使模型能够持续内化不断演化的上下文信息，从而作为一种富表达力、在线演化的状态。

6 结论

我们提出了 In-Place Test-Time Training，这是一个能够解决 TTT 在 LLMs 中关键障碍的实用框架。本文给出了一系列原则性设计选择，包括复用现有 MLP block 的原位机制、高效的 chunk-wise 更新规则，以及一个与语言建模对齐并具备理论依据的目标函数。大量实验验证了，我们的方法不仅能够作为预训练 LLMs 的强力“即插即用”增强，而且在从零训练时也优于强基线。通过为在线适应提供一个可扩展的解决方案，我们的工作朝着更动态、持续学习的 LLM 新范式迈出了有前景的一步。

References

- [1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Martin Cai, et al. Phi-3 technical report: A highly capable language model locally on your phone, 2024. URL <https://arxiv.org/abs/2404.14219>.
- [2] Jimmy Lei Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems*, 2016. URL <https://arxiv.org/abs/1610.06258>.
- [3] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.
- [4] Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. It’s all connected: A journey through test-time memorization, attentional bias, retention, and online optimization, 2025. URL <https://arxiv.org/abs/2504.13173>.
- [5] Ali Behrouz, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni. It’s all connected: A journey through test-time memorization, attentional bias, retention, and online optimization. *arXiv preprint arXiv:2504.13173*, 2025.
- [6] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020. URL <https://arxiv.org/abs/2004.05150>.
- [7] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language, 2019. URL <https://arxiv.org/abs/1911.11641>.
- [8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [9] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024.
- [10] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- [11] Aakanksha Chowdhery and et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022. URL <https://arxiv.org/abs/2204.02311>.
- [12] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv:1803.05457v1*, 2018.
- [13] OpenCompass Contributors. Opencompass: A universal evaluation platform for foundation models. <https://github.com/open-compass/opencompass>, 2023.
- [14] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988. Association for Computational Linguistics, 2019. URL <https://aclanthology.org/P19-1285/>.
- [15] Karan Dalal, Daniel Kocejka, Jiarui Xu, Yue Zhao, Shihao Han, Ka Chun Cheung, Jan Kautz, Yejin Choi, Yu Sun, and Xiaolong Wang. One-minute video generation with test-time training. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 17702–17711, 2025.
- [16] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality, 2024. URL <https://arxiv.org/abs/2405.21060>.
- [17] Tri Dao, Albert Gu, et al. Hungry Hungry Hippos: Towards language modeling with state space models. *arXiv preprint arXiv:2312.00752*, 2023.
- [18] Sri Harsha Dumpala, Chandramouli Sastry, and Sageev Oore. Test-time training for speech, 2023. URL <https://arxiv.org/abs/2309.10930>.

- [19] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Brown, and et al. A mathematical framework for transformer circuits. Transformer Circuits Thread, 2021. URL <https://transformer-circuits.pub/2021/framework/index.html>.
- [20] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling. arXiv preprint arXiv:2101.00027, 2020. URL <https://arxiv.org/abs/2101.00027>.
- [21] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL <https://zenodo.org/records/12608602>.
- [22] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. arXiv preprint arXiv:2012.14913, 2020.
- [23] Team GLM. Chatglm: A family of large language models from glm-130b to glm-4 all tools, 2024. URL <https://arxiv.org/abs/2406.12793>.
- [24] Aaron Grattaffiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024.
- [25] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Realm: Retrieval-augmented language model pre-training. In ICML, 2020.
- [26] Dan Hendrycks, Collin Burns, Steven Basart, Andrew Critch, Jerry Li, Dawn Song, and Jacob Steinhardt. Aligning ai with shared human values. Proceedings of the International Conference on Learning Representations (ICLR), 2021.
- [27] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In International Conference on Learning Representations, 2021. URL <https://arxiv.org/abs/2009.03300>.
- [28] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? arXiv preprint arXiv:2404.06654, 2024. URL <https://arxiv.org/abs/2404.06654>.
- [29] Jinwu Hu, Zhitian Zhang, Guohao Chen, Xutao Wen, Chao Shuai, Wei Luo, Bin Xiao, Yuanqing Li, and Mingkui Tan. Test-time learning for large language models. arXiv preprint arXiv:2505.20633, 2025. URL <https://arxiv.org/abs/2505.20633>. Accepted at ICML 2025.
- [30] Kazuki Irie and Samuel J. Gershman. Fast weight programming and linear transformers: from machine learning to neurobiology, 2025. URL <https://arxiv.org/abs/2508.08435>.
- [31] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7b, 2023. URL <https://arxiv.org/abs/2310.06825>.
- [32] Mahdi Karami and Vahab Mirrokni. Lattice: Learning to efficiently compress the memory. arXiv preprint arXiv:2504.05646, 2025.
- [33] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and Fran  ois Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In Proceedings of the 37th International Conference on Machine Learning, Proceedings of Machine Learning Research. PMLR, 2020. URL <https://arxiv.org/abs/2006.16236>.
- [34] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. In ICLR, 2020.
- [35] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In NeurIPS, 2020.

- [36] Zeman Li, Ali Behrouz, Yuan Deng, Peilin Zhong, Praneeth Kacham, Mahdi Karami, Meisam Razaviyayn, and Vahab Mirrokni. Tnt: Improving chunkwise training for test-time memorization. [arXiv preprint arXiv:2511.07343](#), 2025.
- [37] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. [arXiv preprint arXiv:2412.19437](#), 2024.
- [38] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. [arXiv preprint arXiv:2209.11895](#), 2022. URL <https://arxiv.org/abs/2209.11895>.
- [39] OpenAI. Gpt-4 technical report. [arXiv preprint arXiv:2303.08774](#), 2024.
- [40] Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. Openwebmath: An open dataset of high-quality mathematical web text, 2023.
- [41] Leonid Pekar, Michael Feil, Forrest Moret, Mark Huang, and Tiffany Peng. Llama 3 gradient: A series of long context models, 2024. URL <https://gradient.ai/blog/scaling-rotational-embeddings-for-long-context-language-models>.
- [42] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. [arXiv preprint arXiv:2309.00071](#), 2023.
- [43] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, pages 9355–9366. PMLR, 2021.
- [44] David Silver and Richard S Sutton. Welcome to the era of experience. *Google AI*, 1, 2025.
- [45] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, et al. Paperbench: Evaluating ai’s ability to replicate ai research. [arXiv preprint arXiv:2504.01848](#), 2025.
- [46] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.
- [47] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9229–9248. PMLR, 2020. URL <https://proceedings.mlr.press/v119/sun20b.html>.
- [48] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to (learn at test time): Rnns with expressive hidden states. [arXiv preprint arXiv:2407.04620](#), 2024. URL <https://arxiv.org/abs/2407.04620>.
- [49] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. [arXiv preprint arXiv:2307.08621](#), 2023.
- [50] TogetherAI. Long data collections database, 2024.
- [51] Hugo Touvron, Théo Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, and et al. Llama: Open and efficient foundation language models. [arXiv preprint arXiv:2302.13971](#), 2023.
- [52] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [53] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *ICLR*, 2021.
- [54] Ke Alexander Wang, Jiaxin Shi, and Emily B Fox. Test-time regression: a unifying framework for designing sequence models with associative memory. [arXiv preprint arXiv:2501.12352](#), 2025.

- [55] Yu Wang, Yifan Gao, Xiushi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, Jingbo Shang, and Julian McAuley. Memorylm: Towards self-updatable large language models, 2024. URL <https://arxiv.org/abs/2402.04624>.
- [56] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL <https://arxiv.org/abs/2201.11903>.
- [57] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [58] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023. URL <https://arxiv.org/abs/2312.06635>.
- [59] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. *arXiv preprint arXiv:2412.06464*, 2024.
- [60] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *International Conference on Machine Learning*, pages 56501–56523. PMLR, 2024.
- [61] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024. URL <https://arxiv.org/abs/2406.06484>.
- [62] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [63] Morris Yau, Sharut Gupta, Valerie Engelmayer, Kazuki Irie, Stefanie Jegelka, and Jacob Andreas. Sequential-parallel duality in prefix scannable models, 2025. URL <https://arxiv.org/abs/2506.10918>.
- [64] Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, and Hao Zhou. Memagent: Reshaping long-context llm with multi-conv rl-based memory agent, 2025. URL <https://arxiv.org/abs/2507.02259>.
- [65] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. URL <https://arxiv.org/abs/2502.11089>.
- [66] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, 2019. URL <https://arxiv.org/abs/1905.07830>.
- [67] Tianyuan Zhang, Sai Bi, Yicong Hong, Kai Zhang, Fujun Luan, Songlin Yang, Kalyan Sunkavalli, William T. Freeman, and Hao Tan. Test-time training done right. *arXiv preprint arXiv:2505.23884*, 2025. URL <https://arxiv.org/abs/2505.23884>.

附录

A Proof of theorem 1

For completeness, we first restate the theorem with the precise bounds derived from the assumptions.

Theorem 1 (Logit-wise Effect of LM-Aligned Target v.s. Reconstruction Target (Restated)). *Under the specified setting and assumptions, for a learning rate $\lambda_{lr} > 0$, the expected change in logits $\Delta \ell_n$ after one update step using the **NTP-aligned target** satisfies:*

$$\begin{aligned} \text{(Correct logit increases)} \quad & \mathbb{E}[\Delta \ell_n[v^*]] \geq \lambda_{lr} \cdot c_{norm}^2 \cdot c_{align}, \\ \text{(Other logits almost unchanged)} \quad & |\mathbb{E}[\Delta \ell_n[w]]| \leq \lambda_{lr} \cdot \epsilon \cdot c_{align}, \quad \forall w \neq v^*. \end{aligned} \quad (4)$$

In contrast, for the **reconstruction target**, the expected change in logits is negligible for the correct token:

$$|\mathbb{E}[\Delta \ell_n[v^*]]| \leq \lambda_{lr} \cdot \epsilon \cdot c_{align}. \quad (6)$$

Proof. We begin from the setup defined in section 3.3. The change to the fast weights $\mathbf{W}_{down}$ from all prior context tokens is given by $\Delta \mathbf{W}_{down} = \lambda_{lr} \sum_{t \in \text{prior}} \mathbf{v}_t \mathbf{z}_t^\top$, where we use λ_{lr} to denote the learning rate η for consistency with the theorem statement. The resulting change in the logit for an arbitrary token w at the query position n is:

$$\Delta \ell_n[w] = \mathbf{e}_w^\top (\Delta \mathbf{W}_{down}) \mathbf{z}_n = \lambda_{lr} \sum_{t \in \text{prior}} \mathbf{e}_w^\top (\mathbf{v}_t \mathbf{z}_t^\top) \mathbf{z}_n. \quad (7)$$

Since $\mathbf{e}_w^\top \mathbf{v}_t$ and $\mathbf{z}_t^\top \mathbf{z}_n$ are scalars, we can rearrange the terms to get:

$$\Delta \ell_n[w] = \lambda_{lr} \sum_{t \in \text{prior}} (\mathbf{e}_w^\top \mathbf{v}_t) (\mathbf{z}_t^\top \mathbf{z}_n). \quad (8)$$

To analyze the expected change, we take the expectation over the representations. Applying the linearity of expectation, we have:

$$\mathbb{E}[\Delta \ell_n[w]] = \lambda_{lr} \sum_{t \in \text{prior}} \mathbb{E}[(\mathbf{e}_w^\top \mathbf{v}_t) (\mathbf{z}_t^\top \mathbf{z}_n)]. \quad (9)$$

Per our setup, the target vectors $\mathbf{v}_t$ (e.g., $\mathbf{e}_{x_t}$ or $\mathbf{e}_{x_{t+1}}$) are treated as determined. Thus, we can factor them out of the expectation:

$$\mathbb{E}[\Delta \ell_n[w]] = \lambda_{lr} \sum_{t \in \text{prior}} \mathbf{e}_w^\top \mathbb{E}[\mathbf{v}_t \mathbf{z}_t^\top \mathbf{z}_n]. \quad (10)$$

Now, we invoke Assumption 2. It states that for the unique key position t^* , we have $\mathbb{E}[\mathbf{z}_{t^*}^\top \mathbf{z}_n] = c_{align}$, and for all other prior positions $t \neq t^*$, the updates provide no information gain, which implies $\mathbb{E}[\mathbf{v}_t \mathbf{z}_t^\top \mathbf{z}_n] = \mathbf{0}$. This simplifies the summation to a single term corresponding to the key-value pair (k^*, v^*) at position t^* :

$$\mathbb{E}[\Delta \ell_n[w]] = \lambda_{lr} \cdot \mathbb{E}[(\mathbf{e}_w^\top \mathbf{v}_{t^*}) \cdot (\mathbf{z}_{t^*}^\top \mathbf{z}_n)]. \quad (11)$$

We now analyze this simplified expression for the two target choices.

Case 1: NTP-Aligned Target ($\mathbf{v}_{t^*} = \mathbf{e}_{x_{t^*+1}}$)

First, we consider the logit of the correct token, $w = v^*$. Substituting the target and the token into equation (11) yields:

$$\mathbb{E}[\Delta \ell_n[v^*]] = \lambda_{lr} \cdot \mathbb{E}[(\mathbf{e}_{v^*}^\top \mathbf{e}_{v^*}) \cdot (\mathbf{z}_{t^*}^\top \mathbf{z}_n)]. \quad (12)$$

By **Assumption 1**, token embeddings have a non-trivial magnitude, $\|\mathbf{e}_{v^*}\|^2 \geq c_{norm}^2$. This gives us the lower bound in equation (4):

$$\mathbb{E}[\Delta \ell_n[v^*]] \geq \lambda_{lr} \cdot c_{align} \cdot c_{norm}^2. \quad (13)$$

Next, for any incorrect token $w \neq v^*$, the expected change is $\mathbb{E}[\Delta\ell_n[w]] = \mathbb{E}[(\mathbf{e}_w^\top \mathbf{e}_{v^*}) \cdot (\mathbf{z}_{t^*}^\top \mathbf{z}_n)]$. Taking the absolute value and applying Assumption 1, which states that distinct embeddings are nearly orthogonal ($|\mathbf{e}_w^\top \mathbf{e}_{v^*}| \leq \epsilon$), we obtain the bound in [equation \(5\)](#):

$$|\mathbb{E}[\Delta\ell_n[w]]| \leq \lambda_{\text{lr}} \cdot c_{\text{align}} \cdot \epsilon. \quad (14)$$

Case 2: Reconstruction Target ($\mathbf{v}_{t^*} = \mathbf{e}_{x_{t^*}}$)

Here, we analyze the effect on the correct logit $w = v^*$. The expected change is:

$$\mathbb{E}[\Delta\ell_n[v^*]] = \mathbb{E}[(\mathbf{e}_{k^*}^\top \mathbf{e}_{v^*}) \cdot (\mathbf{z}_{t^*}^\top \mathbf{z}_n)]. \quad (15)$$

In an induction task, the key k^* is distinct from the value v^* . We again invoke Assumption 1 for these distinct tokens. Taking the absolute value gives the bound in [equation \(6\)](#):

$$|\mathbb{E}[\Delta\ell_n[v^*]]| \leq \lambda_{\text{lr}} \cdot c_{\text{align}} \cdot \epsilon. \quad (16)$$

This confirms that the reconstruction target has a negligible expected effect on the logit of the correct answer v^* .

The results from these two cases establish the claims in [theorem 1](#), providing a clear theoretical basis for the superiority of the NTP-aligned objective in the context of in-context learning. This completes the proof. $\square$

B Context Parallel Algorithm for In-Place TTT

For more clarity, we list the pseudocode of the context parallel implementation of our In-Place TTT here in [algorithm 1](#).

Algorithm 1 In-Place TTT with Context Parallelism (Single Layer)

Require: Pre-trained weights θ (incl. $W_{\text{up}}, W_{\text{gate}}, W_{\text{down}}^{(0)}$); Conv1D kernel K ; projection W_{target} ; learning rate η .

- 1: **Input:** Sequence chunks $\{X^{(i)}\}_{i=1}^T$. ▷ Sequence partitioned for Context Parallelism (CP).
- 2: **for all** $i \in \{1, \dots, T\}$ **in parallel do** ▷ Step 1: Compute update deltas.
- 3: $H_i \leftarrow \text{AttentionBlock}(X^{(i)}; \theta)$ ▷ Standard attention, no changes required.
- 4: $U_i, G_i \leftarrow H_i W_{\text{up}}^\top, H_i W_{\text{gate}}^\top$
- 5: $Z_i \leftarrow \phi(G_i) \odot U_i$
- 6: $V_i \leftarrow \text{Conv1D}_K(X_0^{(i)}) W_{\text{target}}$ ▷ Compute NTP-aligned target with causal padding.
- 7: $\Delta W_i \leftarrow V_i^\top Z_i$ ▷ Compute gradient for the fast weight update.
- 8: **end for**
- 9: $\{S_i\}_{i=1}^T \leftarrow \text{CUMSUM}(\{\Delta W_i\}_{i=1}^T)$ ▷ Step 2: Aggregate deltas associatively.
- 10: **for all** $i \in \{1, \dots, T\}$ **in parallel do** ▷ Step 3: Apply updates and compute outputs.
- 11: $W_{\text{down}}^{(i-1)} \leftarrow W_{\text{down}}^{(0)} + \eta S_i$ ▷ Effective weight for chunk i uses updates from chunks $< i$.
- 12: $O_i \leftarrow Z_i (W_{\text{down}}^{(i-1)})^\top$
- 13: **end for**
- 14: **At document boundaries:** Reset fast weights to $W_{\text{down}}^{(0)}$.

C Experiment Details

This appendix provides all details of the experimental settings, datasets, model configurations, and training hyperparameters used for the results presented in Section 4. The following subsections detail the setups

for our three primary sets of experiments: the **continual pre-training** of Qwen3-4B-Base, the **from-scratch pre-training** of models at multiple scales (500M, 1.5B, and 4B), and the **targeted ablation studies**. Our goal is to provide sufficient detail to ensure the reproducibility of our findings.

C.1 Details of Datasets

For the large scale pretraining, continual pretraining, and ablation study, we use the dataset collected by ourselves, we give the details of these datasets as follows.

From Scratch Pretraining Dataset. The pretraining dataset mainly includes general English and Chinese text, along with high knowledge- or reasoning-density data, code, mathematics data, and multilingual text, forming a balanced mixture of linguistic diversity, knowledge and reasoning-rich content, programming material, and mathematical reasoning.

Continual Pretraining Dataset. The continual pretraining dataset is designed to enhance long-context modeling: its short-document portion follows a distribution similar to Pretrain Data, while the long-document portion combines natural data such as books and repository-level code with synthetic data including retrieval-augmented and long-context-QA style constructions, ensuring both consistency with pretraining and coverage of challenging long-context scenarios. The data is organized into subsets with maximum sequence lengths of 32k and 128k for our two-stage training curriculum.

C.2 Details of Training and Evaluation

Training Details. All models are trained on Nvidia H800 GPUs, with the detailed training hyperparameters listed in Tables 4 through 6.

Table 4 Training hyperparameters for 500M and 1.5B models.

Hyperparameter	500M Model	1.5B Model
Optimizer	AdamW	AdamW
Learning Rate	5e-4	3e-4
Batch Size	2M tokens	4M tokens
Weight Decay	0.1	0.1
Gradient Clipping	1.0	1.0
Warmup Steps	1024	1024
Sequence Length	32,768	32,768
Tokens Trained	20B	60B
Sliding Window Size	2,048	4,096

Table 5 Training hyperparameters for 1.7B models and 4B models pretraining

Hyperparameter	value
Optimizer	AdamW
Learning Rate	3e-4
Batch Size	8M tokens
Weight Decay	0.1
Gradient Clipping	1.0
Warm-up Tokens ¹	1.6B
Sequence Length	8,192
Tokens Trained	120B

Evaluation Details. We employ the evaluation framework lm-evaluation-harness [21] to evaluate the models on the common sense reasoning benchmarks and employ the evaluation framework opencompass [13] to evaluate the models on the long context benchmarks. All evaluation are conducted on Nvidia H800 GPUs.

Table 6 Hyperparameters for two-stage continual pre-training.

Hyperparameter	Stage 1 (32k Context)	Stage 2 (128k Context)
Base Model	Qwen3-4B-Base	Qwen3-4B-Base
Optimizer	AdamW	AdamW
Learning Rate	5e-6	5e-6
Weight Decay	0.1	0.1
Sequence Length	32,768	131,072
Tokens Trained	~20B	~15B
RoPE Extension	None	YaRN
Conv Size	5	5

Table 7 Hyperparameters for continual pre-training of LLaMA-3.1-8B and Qwen3-14B-Base.

Hyperparameter	LLaMA-3.1-8B	Qwen3-14B-Base
Optimizer	AdamW	AdamW
Learning Rate	5e-6	5e-6
Weight Decay	0.1	0.1
Sequence Length	32,768	32,768
Tokens Trained	~20B	~20B
RoPE Extension	None	None
Conv Size	5	5

In the evaluation of our continual pretrained Qwen3-4B model, we apply a clipping mechanism at inference time to ensure stable fast-weight updates. Specifically, if the Frobenius norm of an update delta $\|\Delta \mathbf{W}_{\text{down}}^{(i)}\|_F$ exceeds a predefined threshold τ , the delta matrix is rescaled to have norm τ before being applied, i.e., $\Delta \mathbf{W}_{\text{down}}^{(i)} \leftarrow \tau \cdot \Delta \mathbf{W}_{\text{down}}^{(i)} / \|\Delta \mathbf{W}_{\text{down}}^{(i)}\|_F$. This prevents the accumulated updates from growing unboundedly as the sequence length increases, thereby maintaining numerical stability for long-context inference. For all reported evaluations of the Qwen3-4B model, this threshold was set to $\tau = 1\text{e-}5$.

To evaluate the efficiency of our In-Place TTT, we evaluate the prefill throughput and peak memory for sequence length ranging from 8k to 128k. We run the inference for our continual pretrained checkpoints based on Qwen3-4B-Base model. For the setting of sliding window, we set change the attention mechanism of these pretrained checkpoints to sliding window of 1024 tokens manually. We run the inference on Nvidia H800 GPUs with batch size of 1.

C.3 Details of Model Configuration

This section details the architectural configurations of the models used in our experiments. All models are decoder-only Transformer architectures featuring standard components, including SwiGLU activations and Rotary Position Embeddings (RoPE) [46]. The key architectural parameters for all models trained from scratch are summarized in Table 8.

The models trained from scratch employ different attention mechanisms based on their experimental purpose. The 500M, 1.5B utilize sliding-window attention and we list the model configuration in table 8. The 4B-scale experiments and ablation study evaluate two variants: one with full attention and another with sliding-window attention. The backbone architectures for the 4B models and 1.7B models are identical to the Qwen3-4B-Base model and the Qwen3-1.7B-Base model.

For the continual pre-training experiments described in section 4.1, we start directly from publicly available pre-trained models—Qwen3-4B-Base [57], LLaMA-3.1-8B [24], and Qwen3-14B-Base [57]—inheriting their architectures without modification. In experiments featuring our method, the In-Place TTT module is

Table 8 Model architectural configurations for 500M and 1.5B Model.

Parameter	500M	1.5B
Parameters (Approx.)	500M	1.5B
Hidden Size (d_{model})	1024	2048
Num Layers	24	24
Num Attention Heads	8	16
FFN Hidden Size (d_{ff})	3072	6144
Window Size	2048	4096
Vocabulary Size	32,000	32,000
Rope Base	1e6	1e6

integrated into the MLP blocks and applied to every sixth layer. For the ablation studies, this frequency is varied as described in the main paper. The training hyperparameters for Qwen3-4B-Base are listed in [table 6](#), and those for LLaMA-3.1-8B and Qwen3-14B-Base are listed in [table 7](#).

Initialization of In-Place TTT Modules. When integrating In-Place TTT into a pre-trained model for continual training, careful initialization is essential to preserve the model’s pre-trained capabilities at the start of training. Specifically, we initialize the newly introduced TTT components—the Conv1D operator and the projection matrix $\mathbf{W}_{\text{target}}$ —such that the TTT update $\Delta\mathbf{W}_{\text{down}}$ is negligible at initialization, ensuring the model begins from its original pre-trained behavior. Concretely, the depthwise Conv1D (kernel size 5, causal padding, no bias) is zero-initialized, so the target $\hat{\mathbf{V}}$ is zero at initialization. The projection matrix $\mathbf{W}_{\text{target}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$ is initialized as a sparse diagonal matrix, where all off-diagonal entries are zero and the diagonal entries are drawn from $\mathcal{N}(0, \sigma^2)$ with σ being the model’s standard initializer range. This near-zero initialization of both components guarantees that the initial fast-weight update $\eta \hat{\mathbf{V}}_{[i]}^\top \mathbf{Z}_{[i]} \approx \mathbf{0}$, and consequently the effective $\mathbf{W}_{\text{down}}$ remains identical to its pre-trained value. As training progresses, the Conv1D and projection gradually learn to produce meaningful NTP-aligned targets, allowing the TTT mechanism to smoothly emerge without disrupting the pre-trained model.

D Usage of LLMs

During the preparation of this manuscript, LLMs was used to check grammar and improve readability. All authors have reviewed, edited, and take full responsibility for the paper’s final version.